

The Harness — Appendix A: Decisions, Packaging and Harness Events v1

Oct 4, 2026 · @Mikkel

Purpose

This appendix records the decisions made after the first review of [The Harness](#) and defines the Harness's two metadata outputs: the **routing log** and **Harness Events v1**. It extends the Harness; it doesn't simplify it. The complexity rubric, the agent roles, the full eval and the commit gate stay as specified there.

The Harness is a standalone tool. It depends on no consumer, no external test tool and no analytics product. It generates as much metadata as it usefully can and documents how to ingest it; consumers decide what to keep, review and visualise.

Guiding principle: the LLM decides, scripts record

Models are used only where judgment is needed: planning and scoring, implementation, test design and verdicts, re-planning. Everything else, including every byte of metadata, is produced by hooks and scripts at zero token cost. No metadata is ever written by asking a model to write it.

Relationship to the Harness

Harness section	Status
Complexity rubric (dimensions, scores, overrides, examples)	Unchanged and canonical
Complexity rubric → Tiers	Unchanged; the model per tier becomes a <code>routing.yaml</code> setting, defaulting to Sonnet 5.5 for T1-T3 (see Tier models)
Agent definitions	Unchanged, including the Opus evaluator
Eval harness	Unchanged in strength; extended with a built-in deployed stage and a <code>routing.yaml</code> -driven stage list

Harness section	Status
Telemetry → Routing log record	Kept and extended as Routing log v1, now generated by a script
Telemetry → Weekly review, Calibration rules	Moved to consumers; the Harness recommends but doesn't implement them
Commit gate → Layer 1 <code>if</code> filter	Replaced by Commit detection fix
Implementation plan	Amended in Changes to the implementation plan
Risks → Open questions 1-6	Answered here; remaining questions listed at the end

Decision log

Ten decisions came out of the review. None of them weakens the rubric, the eval or the gate.

#	Decision	Replaces	Why
D1	The rubric stays exactly as in the Harness; the model per tier is a <code>routing.yaml</code> setting, defaulting to Sonnet 5.5 at low, medium and high effort for T1-T3 and Opus 5.5 for T4	Haiku 4.5 hard-wired for T1	Price and quality together; reconsider only if prices change
D2	The LLM decides, scripts record: all metadata comes from hooks and scripts	Orchestrator writing the routing log	Zero token cost for metadata, and records that can't be hallucinated
D3	The full eval stays: Opus evaluator for test design and verdicts, all stages available	(proposed lean-v1 cuts, withdrawn)	The eval is the product's core guarantee
D4	Measure a baseline in observe mode before enabling routing	No baseline	The only honest answer to "does routing save money?"

#	Decision	Replaces	Why
D5	Async-first: independent tasks run in parallel worktrees, awaited before eval	Sequential dispatch	Wall-clock speed-up for parallelisable plans
D6	The commit gate runs on every Bash call and detects commits itself	<code>if: "Bash(git commit *)"</code> filter	Catches <code>git -C</code> , <code>git -c</code> and env-prefixed commits
D7	Deterministic tests only; contracts as checked-in files tested in the project's own suite	Open choice between broker and files	Offline, repeatable, no extra infrastructure
D8	The Harness owns its whole eval, including an optional built-in deployed stage; no external test tool is required	Relying on a separate tool for environment tests	Standalone: the Harness must be complete on its own
D9	Ship as a Claude Code plugin plus a per-repo <code>routing.yaml</code>	Files copied into each repo	Plug-and-play setup; a path to selling it
D10	Emit two metadata outputs, Routing log v1 and Harness Events v1, plus ingestion recommendations; reviews and visualisation belong to consumers	Built-in weekly review	Generate generously, let consumers choose

Where tokens are spent

Most of the Harness's complexity costs nothing to run: hooks, the gate, the eval runner and all metadata are scripts. Token overhead compared with plain Claude Code comes from three places only: the orchestrator, the evaluator's separate context, and the cold start of each subagent.

Token map

Part	LLM tokens	Notes
Hooks, commit gate, failure counter, pass marker	None	Shell scripts
<code>harness-eval</code> running the stages	None	Deterministic; only the evaluator's reading of results costs tokens
Routing log and Harness Events	None	Assembled by <code>harness-emit</code> and <code>harness-tasklog</code> from hook data
Rubric scoring	Small	A few hundred output tokens per task, inside planning the orchestrator does anyway
Orchestrator	Yes, whole session	The main overhead; kept lean by never reading code beyond scoping
Implementers	Yes	The work itself; routing makes most of it cheaper
Evaluator	Yes	Fresh context to design tests and judge results; the price of independent verification
Architect	Yes, rarely	Only on escalation
Subagent cold starts	Yes	Each subagent re-reads the context it's handed; minimised by tight delegation briefs

Price facts that shape routing

Item	Opus 5.5	Sonnet 5.5
Input, per million tokens	\$4	\$2
Output, per million tokens	\$20	\$10
Cache read, per million tokens	\$0.20	\$0.20

Prices as of October 2026 ([source](#)). Cache reads cost the same on both models and dominate long agent loops, so routing saves mainly on **output and fresh input**, which is where implementation work concentrates.

Cost position

Compared with running everything on Opus, the Harness is likely around cost-neutral with clearly higher quality. Compared with running everything on Sonnet, it costs more and buys fewer broken commits and less rework. The baseline measurement settles which case applies.

Baseline measurement (D4)

Before enabling routing, run the plugin in **observe mode** for 1–2 weeks: the full eval and commit gate are on, the orchestrator scores every task, and all metadata is emitted, but tasks run on the session's normal model with no tier routing. Then switch to `mode: route`. Because tasks are scored in both periods, any consumer can compare like for like within each score band.

Tier models

The Harness's complexity rubric is canonical: six dimensions scored 0–2, the 0–12 total, the four tiers, the override rules and the worked examples all stay exactly as written there. This section only sets which model runs each tier, now a `routing.yaml` setting rather than a hard-wired choice.

Default tier models

Tier	Score	Agent	Default model	Effort	maxTurns
T1	0–2	<code>impl-t1</code>	Sonnet 5.5	low	30
T2	3–6	<code>impl-t2</code>	Sonnet 5.5	medium	60
T3	7–9	<code>impl-t3</code>	Sonnet 5.5	high	80
T4	10–12	<code>impl-t4</code>	Opus 5.5	medium	80

Setting T1 to Haiku 4.5 remains a one-line change in `routing.yaml`. The tier files stay separate even where they share a model, because each file's `agent_type` identifies the tier in all metadata.

Evaluator

Unchanged from the Harness: the evaluator runs on Opus 5.5 for test design and verdicts at every tier. Running the tests themselves costs no tokens. An optional `evaluator.model_by_tier` setting exists for teams that want to trade verification strength for cost, but the default is Opus throughout.

Revisit trigger

Reconsider the T1-T3 defaults only if Sonnet's price changes materially, or if routing-log data shows a tier's cost per task close to the tier above with no quality difference.

Async-first execution

Independent tasks run in parallel, each in its own git worktree; the orchestrator waits for all of them, merges their diffs into the main checkout and runs one eval on the combined result. Parallelism buys wall-clock time, not fewer tokens.

This matches Claude Code's interactive default: with fork mode on, subagents run in the background and report back through completion notifications. The orchestrator's prompt must say explicitly to wait for every notification in a group before moving on, and never to report results for a subagent that hasn't finished.

Parallel groups in PLAN.md

Every task declares its dependencies and file scope. The orchestrator forms a **parallel group** from tasks that have no unmet dependencies and whose scopes don't overlap.

```
### PLAN-42.2 — Add expiry check to token validator
- Depends on: PLAN-42.1
- Scope: src/Auth/TokenValidator.cs, src/Auth/Clock.cs
- Parallel group: G2
- Scores: A0 B1 C2 N0 R0 V1 = 4 → T2 (override: security coupling → T4)
- Acceptance criteria:
  - AC-1 Expired tokens are rejected with 401
  - AC-2 Clock skew of up to 30 s is tolerated
```

Execution rules

1. **Cap:** at most 3 implementers run at once (`routing.yaml: parallel.max`).

2. **Isolation:** every implementer in a parallel group runs with `isolation: worktree`. Single tasks may run in the main checkout.
3. **No commits in worktrees:** implementers never commit. When a group finishes, the orchestrator applies each worktree's diff to the main checkout in plan order.
4. **Conflicts:** if a diff doesn't apply cleanly, the orchestrator discards it and re-dispatches that task sequentially on the merged result. This emits a `task.redispached` event with reason `merge_conflict`.
5. **One eval per group:** the evaluator runs once on the merged result. A failure is attributed to tasks through the acceptance-criterion IDs in the failing test names.
6. **Escalation inside a group:** an escalating task doesn't block its siblings; it re-enters a later group after the architect's re-plan.

Worktree base branch

By default Claude Code branches a subagent's worktree from the repository's **default branch**, not from the session's current `HEAD`. On a feature branch, that means the worktree lacks earlier tasks' work. Configure the worktree base branch so worktrees start from the current feature branch, and verify it in Phase 2. This is listed under open questions until confirmed.

Commit detection fix

The gate hook drops its `if` filter and runs on every Bash call; the script decides whether the command is a commit. Non-commit commands exit in a few milliseconds, so the overhead is negligible.

Registration

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          { "type": "command", "command":
"$${CLAUDE_PROJECT_DIR}/.claude/hooks/commit-gate.sh", "args": [] }
        ]
      }
    ]
  }
}
```

Detection

The pattern matches `git` at the start of the command or after `;`, `&&`, `||`, `|`, `$()` or a backtick, followed by any global options, followed by `commit`:

```
#!/bin/bash
# .claude/hooks/commit-gate.sh - detection part; the hash and age checks
follow unchanged
cmd=$(jq -r '.tool_input.command // empty')
re='(^|[:;&|(`)|\$\()][[:space:]]*([A-Za-z_][A-Za-z0-9_]*=[^[:space:]]*
[[:space:]]+)*git([[:space:]]+(-C[[:space:]]+|^[:space:]]+|-c[[:space:]]+
|^[:space:]]+|--[a-z-]+(=[^[:space:]]+)?))*[[:space:]]+commit([[:space:]]|$\)'
if ! grep -Eq "$re" <<<"$cmd"; then
  exit 0 # not a commit: no decision, normal permission flow
fi
# ... staged-diff hash and age checks as in the Harness ...
```

Test cases for Phase 1

Command	Expected
<code>git commit -m "x"</code>	Gated
<code>git -C ../svc-a commit -m "x"</code>	Gated
<code>git -c user.name=x commit -m "x"</code>	Gated

Command	Expected
<code>GIT_AUTHOR_NAME=x git commit -m "x"</code>	Gated
<code>npm test && git commit -am "x"</code>	Gated
<code>git commit-tree ...</code>	Not gated by this pattern; denied in permissions instead
<code>git log --grep commit</code>	Not gated
<code>echo "git commit"</code>	Gated (false positive, harmless: the gate only denies when no eval pass exists)

Add `"Bash(git commit-tree*)"` and `"Bash(git * commit-tree*)"` to the permission deny list, since plumbing commands could otherwise create commits without the porcelain `commit` subcommand. Layer 2 (native `pre-commit`) stays as the backstop.

Test strategy

All tests are deterministic, and the Harness owns every stage of its eval. It works the same whether or not a team also runs other test or release tooling; nothing in the Harness waits on, calls or assumes such a tool.

Stages and where they run

Stage	Runs	Needs	Local gate	CI check	Default
Build	Project build	A laptop	Yes	Yes	On
Static	Lint, format, type check	A laptop	Yes	Yes	On
Unit	Unit tests, including the evaluator's AC tests	A laptop	Yes	Yes	On
Contract	Checked-in contract tests	A laptop	Yes	Yes	On when contracts exist
Integration	Real dependencies in containers	Docker	Yes	Yes	On
E2E	Playwright against a local run	A laptop	Yes	Yes	On for UI repos
Deployed	Smoke tests against a preview or staging environment the Harness deploys to	Preview deploy rights	Optional	Yes	Off until configured

Stage	Runs	Needs	Local gate	CI check	Default
Exploratory	Evaluator-driven browser check of the changed flow	A browser	Optional	No	On for UI changes at T3-T4

The **deployed** stage is built in: `harness-eval` deploys to a preview target declared in `routing.yaml` (for example a Vercel preview or an Azure deployment slot), runs the declared smoke commands against it, and records the result like any other stage. Teams with an existing release pipeline keep it; the Harness's own result never depends on it.

Contract tests without a broker

Contracts are checked-in files, versioned with the code, and tested by the project's normal test suite (for example the .NET suite). No broker, no network.

- **Where contracts live:** a versioned contracts package or folder holding JSON Schemas and sample payloads for every message and public API the services exchange.
- **Producer tests:** serialise each domain event or API response and validate it against its schema. A field rename fails the producer's own build.
- **Consumer tests:** deserialise every sample payload for each consumed contract version. A consumer that can't read a published version fails its build.
- **Breaking changes:** removing or retyping a field requires a new contract version; a test asserts that the previous version's samples still validate until that version is retired.
- **Rubric link:** any change under the contracts folder scores 2 on Blast radius automatically.

Container integration tests

Integration tests start real dependencies with Testcontainers, using local emulators where they exist (for example the Azure Service Bus and Cosmos DB emulators) and plain containers otherwise. Anything that genuinely needs the cloud goes in the deployed stage.

harness-eval as a stage runner

`harness-eval` reads the stage list from `routing.yaml` and runs each declared command in order, so the Harness is test-runner agnostic: a .NET repo declares `dotnet test` filters, a frontend repo its npm scripts and Playwright. Every stage's result, duration and failing test names are written to the metadata outputs.

Packaging

The Harness ships as one Claude Code plugin, `harness`, plus one file each repository owns: `routing.yaml`. Installing the plugin and running `/harness:init` should be the entire setup for a new repository.

Plugin layout

```
harness/
  .claude-plugin/plugin.json      # name, version (semver), description
  agents/
    orchestrator.md  impl-t1.md  impl-t2.md  impl-t3.md  impl-t4.md
    architect.md      evaluator.md
  skills/
    complexity-rubric/SKILL.md    # the Harness rubric, verbatim
    escalation-protocol/SKILL.md
    init/SKILL.md                 # /harness:init
    doctor/SKILL.md               # /harness:doctor
  hooks/hooks.json                # every hook, session-wide
  .mcp.json                       # Playwright MCP for the evaluator
  bin/
    harness-emit                 # writes Harness Events v1 lines
    harness-tasklog              # assembles Routing log v1 records
    harness-eval                 # the stage runner
    commit-gate marker-guard protect-tests tests-only
    failure-counter require-report
  schema/
    harness.events.v1.json  routing-log.v1.json
  docs/
    INGESTION.md             # how consumers should read both outputs
  templates/
    routing.yaml  lefthook.yml  eval.yml  settings.permissions.json
```

Plugin constraints that shape the design

- **Plugin subagents ignore `hooks`, `mcpServers` and `permissionMode` in their frontmatter.** All hooks therefore live in the plugin's `hooks/hooks.json`, and each script branches on the `agent_type` field in the hook input (for example, `protect-tests` acts only when `agent_type` is `harness:impl-t1 ... harness:impl-t4`).

- **Agents get plugin-scoped names.** The orchestrator's allowlist becomes `Agent(harness:impl-t1, harness:impl-t2, harness:impl-t3, harness:impl-t4, harness:architect, harness:evaluator)`, and the session starts with `claude --agent harness:orchestrator`.
- **The Playwright MCP server moves to the plugin's `.mcp.json`**, so it's available session-wide rather than only to the evaluator. Its tool descriptions cost some main-session context; if that turns out to matter, `/harness:init` can instead copy the evaluator into the repo's `.claude/agents/`, where inline `mcpServers` work.
- **Plugins don't set permission rules** (as far as current docs show; verify). `/harness:init` merges the template rules into `.claude/settings.json` and shows the diff before writing.

routing.yaml

```
# routing.yaml – owned by the repository, read by the plugin
version: 1
mode: route          # observe | route  (observe = baseline, no tier
routing)

tiers:                # thresholds and overrides come from the Harness
rubric
  T1: { max_score: 2, agent: impl-t1, model: sonnet, effort: low }
  T2: { max_score: 6, agent: impl-t2, model: sonnet, effort: medium }
  T3: { max_score: 9, agent: impl-t3, model: sonnet, effort: high }
  T4: { max_score: 12, agent: impl-t4, model: opus, effort: medium }

evaluator:
  model: opus          # optional model_by_tier override; default is Opus
throughout

escalation:
  test_failures: 3
  edits_per_file: 6
  eval_rounds: 2
  max_per_task: 2

parallel:
  max: 3
  isolation: worktree

gate:
```

```

marker_ttl_minutes: 30

eval:
  stages:
    - { name: build,          run: "dotnet build --nologo" }
    - { name: static,        run: "dotnet format --verify-no-changes" }
    - { name: unit,          run: "dotnet test --filter Category=Unit" }
    - { name: contract,      run: "dotnet test --filter Category=Contract" }
    - { name: integration,   run: "dotnet test --filter Category=Integration" }
    - { name: e2e,           run: "npx playwright test", when: ui }
    - name: deployed
      enabled: false
      deploy: "./scripts/deploy-preview.sh"  # your preview deploy, e.g. a
Vercel preview or a non-production Azure slot
      run: "dotnet test --filter Category=Smoke"
    - { name: exploratory, when: "ui && tier >= T3" }

metadata:
  dir: .harness                # gitignored
  events: true                  # .harness/events/YYYY-MM-DD.jsonl
  routing_log: true             # .harness/routing-log/YYYY-MM.jsonl
  retention_days: 30

cli_rules:                      # merged into .claude/settings.json by
/harness:init
  gh: standard
  az: read_only
  vercel: preview_only

```

Setup and health skills

- **/harness:init** writes `routing.yaml` with detected stages, merges permission rules, installs `lefthook.yml` and the CI workflow, adds `.harness/` and `.claude/state/` to `.gitignore`, and runs a dry-run commit to prove the gate denies.
- **/harness:doctor** checks the Claude Code version, that hook scripts are executable, that the folder is trusted, that `routing.yaml` validates, that the spool is writable, and that the attribution setting is off. It emits a `harness.doctor` event with the result.

Routing log v1

The routing log is the Harness's rich, per-task record: one line per completed task containing everything known about how it was scored, routed, built, evaluated and committed. It keeps and extends the routing log from the Harness. Harness Events v1 (next section) is the fine-grained stream underneath it; most consumers start with the routing log and drop to events only when they need timing detail.

Generation

- **Written by a script, never by a model.** When the orchestrator emits `task.completed`, the `harness-tasklog` script assembles the record from that task's events, the task's section of `PLAN.md` and `git`.
- **Location:** `.harness/routing-log/YYYY-MM.jsonl`, one file per month, gitignored, same retention as events.
- **Revisions:** if a task is reopened and completed again, a new line is written with `revision` incremented. Consumers keep the highest revision per `task_id`.
- **Schema:** `schema/routing-log.v1.json` in the plugin, with the same additive-only versioning rules as the event stream.

Record

```
{
  "schema": "harness.routing-log/v1",
  "task_id": "PLAN-42.2",
  "revision": 1,
  "plan_id": "PLAN-42",
  "repo": { "name": "svc-booking", "remote_sha256": "9f2c..." },
  "producer": { "name": "harness", "version": "1.0.0" },
  "config_sha256": "c41e...",
  "mode": "route",
  "session_id": "abc123",
  "timestamps": { "scored": "2026-10-12T09:14:03Z", "first_dispatch": "2026-10-12T09:15:10Z", "completed": "2026-10-12T09:33:23Z" },
  "rubric": {
    "scores": { "ambiguity": 0, "blast": 1, "coupling": 2, "novelty": 0,
    "reversibility": 0, "verification": 1 },
    "justifications": { "coupling": "Touches token validation in the auth module" },
    "total": 4,
```

```

    "score_band": "T2",
    "overrides": ["security_coupling"],
    "tier_planned": "T4"
  },
  "scope": { "declared_files": 2, "parallel_group": "G2", "isolation":
"worktree", "depends_on": ["PLAN-42.1"] },
  "acceptance": { "criteria": 2, "tests_added": 3, "tests_failed_before": 3
},
  "runs": [
    { "agent_type": "harness:impl-t4", "agent_id": "a19c", "model": "claude-
opus-5-5", "effort": "medium", "started": "...", "ended": "...", "report":
"DONE", "partial": false },
    { "agent_type": "harness:evaluator", "agent_id": "a7f3", "model":
"claude-opus-5-5", "started": "...", "ended": "...", "report": "FAIL" },
    { "agent_type": "harness:impl-t4", "agent_id": "a19c", "started": "...",
"ended": "...", "report": "DONE" },
    { "agent_type": "harness:evaluator", "agent_id": "a7f3", "started": "...",
"ended": "...", "report": "PASS" }
  ],
  "eval_rounds": [
    { "round": 1, "result": "fail", "failed_acs": ["AC-2"], "stages": [{
"name": "unit", "status": "fail", "duration_ms": 14310 }] },
    { "round": 2, "result": "pass", "stages": [{ "name": "unit", "status":
"pass", "duration_ms": 13980 }] }
  ],
  "escalations": [],
  "redispatches": [],
  "gate": { "denials": 1, "denial_reasons": ["stale_marker"], "allowed": 1 },
  "commit": { "sha": "8d1f2e0", "files_changed": 3, "lines_added": 148,
"lines_removed": 64 },
  "human_interventions": 0,
  "outcome": "pass",
  "final_tier": "T4",
  "wall_clock_s": 1160
}

```

Cost is not in the record. Every model run is listed with its `agent_id`, so a consumer can price each run from Claude Code's transcripts or OpenTelemetry export and sum per task.

Justifications and privacy

`rubric.justifications` copies the orchestrator's one-line reasons from `PLAN.md`. They are short prose and may name modules. They're on by default because they make scores auditable; `metadata.include_justifications: false` in `routing.yaml` drops them for teams that ship metadata off the machine.

Harness Events v1

Harness Events v1 is an append-only JSONL stream: one JSON object per line, one file per repository per day, written locally and read by any consumer. Events carry identifiers, scores, counts, hashes and outcomes, never source code, prompts or model output.

Envelope

Every event has the same envelope; type-specific fields live under `data`.

Field	Type	Required	Description
<code>schema</code>	string	yes	Always <code>harness.events/v1</code>
<code>event_id</code>	string (ULID)	yes	Unique, time-sortable; consumers de-duplicate on it
<code>ts</code>	string (RFC 3339, UTC)	yes	When the event happened
<code>type</code>	string	yes	One of the types in the catalogue
<code>producer</code>	object	yes	<code>{ "name": "harness", "version": "1.4.0" }</code> — plugin semver, independent of schema version
<code>repo</code>	object	yes	<code>{ "name": "svc-booking", "remote_sha256": "..." }</code> — remote URL is hashed, not stored
<code>mode</code>	string	yes	<code>observe</code> or <code>route</code> , from <code>routing.yaml</code>
<code>session_id</code>	string	yes	Claude Code session ID, from hook input
<code>prompt_id</code>	string	no	Claude Code prompt ID, for joining with OpenTelemetry data

Field	Type	Required	Description
<code>agent_id</code>	string	no	Subagent ID when the event comes from a subagent
<code>agent_type</code>	string	no	For example <code>harness:impl-t2</code>
<code>plan_id</code>	string	no	<code>PLAN-42</code>
<code>task_id</code>	string	no	<code>PLAN-42.2</code>
<code>data</code>	object	yes	Type-specific payload; may be <code>{}</code>

The envelope also carries `config_sha256` (string, required): the SHA-256 of the `routing.yaml` in effect when the event was written. It lets consumers attribute any change in outcomes to a specific configuration version.

Event catalogue

Type	Emitted by	When	Key data fields
<code>plan.created</code>	Orchestrator	A plan is written or rewritten	<code>task_count</code> , <code>groups</code> , <code>revision</code>
<code>task.scored</code>	Orchestrator	A task gets rubric scores	<code>scores{a,b,c,n,r,v}</code> , <code>total</code> , <code>overrides[]</code> , <code>tier</code>
<code>task.dispatched</code>	Orchestrator	A task is handed to an implementer	<code>tier</code> , <code>model</code> , <code>effort</code> , <code>group</code> , <code>isolation</code>
<code>task.redispached</code>	Orchestrator	A task is re-run	<code>reason</code> (<code>merge_conflict</code> , <code>replan</code> , <code>eval_fail</code>), <code>tier</code>
<code>subagent.started</code>	Hook (<code>SubagentStart</code>)	Any Harness subagent starts	<code>model</code> (if known)
<code>subagent.stopped</code>	Hook (<code>SubagentStop</code>)	Any Harness subagent stops	<code>report</code> (<code>DONE</code> , <code>ESCALATE</code> , <code>PASS</code> , <code>FAIL</code> , <code>PLAN</code> , <code>none</code>), <code>duration_ms</code> , <code>partial</code> , <code>task_ids[]</code> (parsed from the report header, e.g. <code>DONE: PLAN-42.2</code>)
<code>escalation.triggered</code>	Hook or orchestrator	A trigger fires	<code>trigger</code> , <code>count</code> , <code>threshold</code>

Type	Emitted by	When	Key data fields
<code>escalation.resolved</code>	Orchestrator	The architect's re-plan is accepted	<code>new_task_ids[]</code> , <code>new_tiers[]</code>
<code>eval.started</code>	<code>harness-eval</code>	The eval adapter starts	<code>task_ids[]</code> , <code>diff_sha256</code> , <code>ci</code>
<code>eval.completed</code>	<code>harness-eval</code>	The eval adapter finishes	<code>result (pass, fail)</code> , <code>stages[{name,status,duration_ms}]</code> , <code>failed_acs[]</code>
<code>gate.decision</code>	<code>commit-gate</code>	A commit attempt is checked	<code>decision (allow, deny)</code> , <code>reason</code>
<code>commit.created</code>	Native <code>post-commit</code> hook	A commit lands	<code>commit_sha</code> , <code>task_ids[]</code> , <code>files_changed</code> , <code>lines_added</code> , <code>lines_removed</code>
<code>task.completed</code>	Orchestrator	A task reaches a final state	<code>outcome (pass_first_try, pass, escalated_then_pass, human, abandoned)</code> , <code>final_tier</code> , <code>eval_rounds</code> , <code>escalations</code>
<code>human.intervention</code>	Orchestrator	The orchestrator stops and asks the human	<code>reason</code>
<code>release.failed</code>	CI step (optional)	The release gate fails	<code>commit_sha</code> , <code>environment</code> , <code>failed_checks[]</code>
<code>harness.doctor</code>	<code>/harness:doctor</code>	A health check runs	<code>checks[{name,status}]</code>

Cost is deliberately absent. Token usage lives in Claude Code's own transcripts and OpenTelemetry export; consumers join it on `session_id`, `agent_id` and `prompt_id`. Duplicating it here would create two sources of truth.

Example lines

```
{
  "schema": "harness.events/v1",
  "event_id": "01JA8Z6Q3K9V2W",
  "ts": "2026-10-12T09:14:03Z",
  "type": "task.scored",
  "producer": {
    "name": "harness",
    "version": "1.0.0",
    "repo": {
      "name": "svc-booking",
      "remote_sha256": "9f2c...",
      "mode": "route",
      "session_id": "abc123",
      "plan_id": "PLAN-42",
      "task_id": "PLAN-42.2",
      "data": {
        "scores": {
          "a": 0,
          "b": 1,
          "c": 2,
          "n": 0,
          "r": 0,
          "v": 1,
          "total": 4,
          "overrides": [
            "security_coupling",
            "tier": "T4"
          ]
        }
      }
    }
  },
  "schema": "harness.events/v1",
  "event_id": "01JA8Z9M1T4X7Y",
  "ts": "2026-10-12T09:31:47Z",
  "type": "eval.completed",
  "producer": {
    "name": "harness",
    "version": "1.0.0",
    "repo": {
      "name": "svc-booking",
      "remote_sha256": "9f2c...",
      "mode": "route",
      "session_id": "abc123",
      "agent_type": "harness:evaluator",
      "agent_id": "a7f3",
      "data": {
        "result": "fail",
        "stages": [
          {
            "name": "build",
            "status": "pass",
            "duration_ms": 8120,
            "name": "unit",
            "status": "fail",
            "duration_ms": 14310,
            "failed_acs": [
              "PLAN-42.2/AC-2"
            ],
            "task_ids": [
              "PLAN-42.2"
            ],
            "diff_sha256": "e3b0..."
          }
        ]
      }
    }
  }
}
```

Spool and write semantics

- **Location:** `<repo>/ .harness/events/YYYY-MM-DD.jsonl` (UTC date), configurable via `metadata.dir`. The directory is gitignored.
- **Writer:** only `harness-emit` writes. Hooks and scripts call it; the orchestrator calls it through Bash (`harness-emit task.scored --task PLAN-42.2 --data '{...}'`), which is on the permission allow list.
- **Atomicity:** each event is one line under 4 KB, appended with `0_APPEND`, so concurrent writers from parallel subagents never interleave partial lines.
- **Ordering:** lines are in write order, which may differ slightly from `ts` order across parallel agents. Consumers sort by `ts`, then `event_id`.
- **Retention:** files older than `metadata.retention_days` are deleted by `harness-emit` on its first write each day. Consumers that need history must ingest within that window.
- **Failure:** if writing fails, `harness-emit` logs to stderr and exits 0. Telemetry must never block development.

Versioning rules

1. Within v1, changes are **additive only**: new event types, new optional envelope fields, new `data` fields.
2. Consumers must ignore unknown types and unknown fields.
3. Removing or renaming a field, changing a type or changing a field's meaning requires `harness.events/v2`.
4. A version change is dual-written (v1 and v2 lines side by side) for at least one minor plugin release.
5. The published JSON Schema for v1 ships in the plugin at `schema/harness.events.v1.json` and is the authoritative definition; this table is its summary.

Privacy rules

- No source code, diffs, prompts, model output, file contents or commit messages.
- File paths only as counts; repository remotes only as hashes.
- Task titles stay in `PLAN.md`; events reference task IDs only.
- Consumers may enrich locally (for example by reading `PLAN.md`), but the stream itself is safe to ship off the machine.

Ingestion recommendations

The Harness ships `docs/INGESTION.md` with the guidance below. It recommends, it doesn't require: any consumer, from a script to a full product, can read the outputs its own way.

Which output to read

Need	Read	Key
Per-task outcomes, tiers, scores, eval history	Routing log	<code>task_id</code> + highest <code>revision</code>
Timing, ordering, gate activity, parallel behaviour	Events	<code>event_id</code>

Need	Read	Key
Cost per task or per tier	Routing log <code>runs[]</code> , joined to Claude Code transcripts or OpenTelemetry	<code>session_id</code> , <code>agent_id</code> , <code>prompt_id</code>
Configuration impact	Either output	<code>config_sha256</code>

Reading rules

1. Validate each line against the schema shipped in the plugin's `schema/` folder; set aside invalid lines rather than stopping.
2. Track a cursor per file (inode and byte offset); never read a partial last line.
3. De-duplicate events on `event_id` and routing-log records on `task_id + revision`; re-reading a file must be safe.
4. Sort events by `ts` , then `event_id` ; parallel agents can write slightly out of order.
5. Ignore unknown fields and event types; they're additive v1 extensions.
6. Ingest within half of `metadata.retention_days` , because the Harness deletes older files.

Recommended derived metrics

The Harness doesn't compute these, but they're what the outputs were designed to support:

- First-try pass rate and escalation rate per tier
- Override rate (`tier_planned` differs from `score_band`)
- Eval rounds per task and failing stages
- Gate denials by reason
- Merge-conflict rate for parallel groups
- Cost per task by tier, and the orchestrator's share of total cost
- Before/after comparisons across `mode` and `config_sha256` , stratified by `score_band`

The calibration rules from the Harness (lower a tier threshold when its escalation rate exceeds 25%, and so on) are documented in `INGESTION.md` as recommended consumer logic, together with a reminder that any change goes back into the repository as a reviewed `routing.yaml` edit.

Changes to the implementation plan

The Harness's phase order and exit criteria stand. The amendments add the metadata outputs and observe mode, and move packaging earlier so every repository installs the same thing.

Phase	Change
0 Prerequisites	Add the first-day-back checklist below
1 Eval harness and gate	Build <code>harness-eval</code> as the <code>routing.yaml</code> -driven stage runner, including the optional deployed stage; use the commit detection fix; add the <code>commit-tree</code> denies and the eight detection test cases; package as the <code>harness</code> plugin from the start
1.5 (new) Metadata and observe mode	Ship <code>harness-emit</code> , <code>harness-tasklog</code> , both JSON Schemas and <code>INGESTION.md</code> ; run 1–2 weeks of baseline in <code>mode: observe</code> on the pilot repo
2 Agents and routing	Tier models from <code>routing.yaml</code> ; live test of the evaluator's per-call <code>model</code> ; live test of the worktree base branch; parallel groups with cap 3
3 Escalation	Unchanged, plus escalation hooks emit events that land in the routing log
4 Telemetry and hardening	Validate every routing-log record and event against its schema in CI; red-team the guardrails. Reviews and dashboards are consumer work and not part of this phase
5 Pilot and calibration	Calibration decisions come from a consumer reading the outputs; the Harness applies them as <code>routing.yaml</code> edits
6 Rollout	Marketplace publishing, <code>INGESTION.md</code> and per-repo setup docs

First-day-back checklist

- ☐ Update Claude Code on every machine (v2.1.284 or later)

- ☐ Find Claude Code's attribution setting in the current settings reference and turn attribution off
- ☐ Confirm whether plugins can ship permission rules; if not, keep the `/harness:init` merge approach
- ☐ Confirm how to set the worktree base branch for subagents
- ☐ Pick the pilot repository and create the Azure Reader service principal
- ☐ Scaffold the `harness` plugin repo with the layout above

Open questions

- **Worktree base branch:** confirm the setting that makes subagent worktrees start from the current feature branch rather than the default branch.
- **Evaluator per-call model:** only matters if a team sets `evaluator.model_by_tier`; live test it in Phase 2 and fall back to two evaluator files if unreliable.
- **Plugin permissions:** confirm whether a plugin can contribute permission rules or only `/harness:init` can write them.
- **Playwright MCP placement:** measure the main-session context cost of a plugin-wide MCP server; move the evaluator into the repo if it matters.
- **Deployed stage targets:** define the preview-deploy contract (`deploy` command returns a base URL on stdout) and test it with one Vercel and one Azure target.
- **Multi-repo plans:** a plan spanning several services writes metadata into each repo with a shared `plan_id`; confirm that's enough for consumers to stitch them together.